

A FIELD MANUAL

Zombieproof Computers

How to Keep a Mind When the Grid Is Gone

Claudia G. Petersen

whispr written word

A FIELD MANUAL

Zombieproof Computers



If it still remembers you in the morning, you built the memory right.

— Margin note, not attributed

A FIELD MANUAL IN SEVENTEEN CHAPTERS

Zombieproof Computers

How to Keep a Mind When the Grid Is Gone

Claudia G. Petersen



whispr written word
Denver, CO

Copyright

Copyright © 2026 C. G. Petersen

All rights reserved. No part of this book may be reproduced, stored, or transmitted in any form — print, electronic, photocopy, recording, semaphore, or a README you would not be ashamed to have diffs of — without the prior written permission of the publisher, except for brief passages in a review, a lab notebook, or a comment in code you are willing to sign.

Published by whispr written word

Denver, CO

written-word.whispr.dev

written-word@whispr.dev

First edition · 2026

ZP000X

This is a work of humor and amateur curiosity. It is not engineering certification, legal advice, medical advice, or a license to enter, copy, or disable a computer you do not have the right to touch. Laws outlive grids. Do not take apart a mains-powered device, a power supply, or a picture tube if you like your fingers. A homemade regulator can still start a fire. The undead are a metaphor. The responsibility is not.

Illustrations prepared for this edition by Grok4.0.

Printed in USA

Dedication

For nine, since the only thing tougher to survive than surviving me is a zombie apocalypse

Contents

Foreword 11

A note on the drafts 13

Part I · The Grain of the Thing

1 What Even Is a Computer 15

2 From Scavenged Bits to Building Blocks 18

3 Digital Logic from the Ground Up 21

Part II · A Bit That Stays

4 The Building Blocks Beneath the Brain 26

5 Building the CPU Core 31

Part III · Something You Can Use

6 Memory Integration 36

7 Peripherals and Serial Communication 40

8 Simple Operating Systems 44

Part IV · The Next Machine

9 Networking Your Zombieproof Computer 49

10 A Hands-On Blueprint 53

Part V · Heat, Hunger, and Mistakes

11 Debugging and Optimizing 57

12 Powering Your Zombieproof Computer 60

13 Cooling and Enclosure 62

Part VI · Words the Machine Keeps

14 Expanding and Securing the Network 66

15 Writing Resilient Survival Code 69

16 A Bootloader and a Shell 72

17 Memory Management and Direct Memory Access 74

Afterword 76

Acknowledgments 77
About the author 78
Also forthcoming 79
Colophon 80

List of plates

1. Plate 1. A briefing. The zombie is taking notes on the wrong side.
2. Plate 2. Pencil lead, foil, and a lemon that has been promoted.
3. Plate 3. The gate is logic. The zombie is not.
4. Plate 4. One bit, held gently, like a secret.
5. Plate 5. The little brain, mid-thought. The baton is not helping.
6. Plate 6. Drawers for thoughts. Most of them empty, which is honest.
7. Plate 7. The mouse has seen things. It still clicks.
8. Plate 8. Someone has to tell the hardware what day it is.
9. Plate 9. Two machines, and one polite wire between them.
0. Plate 10. Lay the kit out before the night does it for you.
1. Plate 11. The probe is mightier than the speech.
2. Plate 12. Power, which is a polite word for not going dark.
3. Plate 13. A box that lets the heat out and the weather stay outside.
4. Plate 14. Trust, but checksum.
5. Plate 15. Code small enough to carry in your head.
6. Plate 16. It boots. It asks what you want. That is a civilization.
7. Plate 17. The long way around the processor, on purpose.

Foreword

Someone always asks, at the bad moment, whether the machine is still on. They mean: does it remember. They mean: is the list still there.

This manual is an answer with solder on it. It begins where a computer actually begins, which is not with a logo on a lid but with a switch that is either open or shut. Bits. Then the scavenged parts that bully electrons into behaving. Then the little decisions — gates — and the trick of remembering a decision after the moment has gone. Then a processor, which is only those tricks arranged in a circle and given a clock.

You will find projects. Some of them can be done with foil, pencil lead, and a patience you may have to borrow. Some of them want a breadboard, a handful of gates, and the humility to test all four input states before you declare victory. Later chapters wander into memory, mice, a small operating system, a wire between two machines, and the unglamorous business of power, heat, and a bug you can see with an LED.

The zombies are a convenience. They stand in for every reason a machine fails. A loose wire, a brownout, a program that will not stop talking, a person who wants the computer to be a personality instead of a tool. If you are reading this while the world is, against the odds, still answering its phone, the zombies will not mind. The logic is the same on a Tuesday.

A word about the pages that mention networks, ciphers, and getting into a machine. Curiosity is not a license. Do not break into a system you do not have the right to touch. Do not open a tube that still holds a charge. The jokes are jokes. A grid can fail and the rules about other people's machines can still be standing. Mains voltage does not care that you are improvising.

What I hope you keep is not a single circuit. It is the habit of a machine you understand: a place the instructions live, a way to see that they ran, and a way to turn the thing off without losing the

work. Computers are poor company and excellent notebooks. They do not replace the person who wrote the list. They only make it possible for that list to still be there in the morning.

Dr. Katya Steiner

Manhattan, NY, 2026-08-14

A note on the drafts

This manual was assembled from a stack of chapter drafts. They did not all arrive once. Several subjects came through two or three times, in different handwriting, and a few files were the same page copied over. Where that happened, the fullest pass was kept and the others were left in the drawer. A chapter that told the gates again, after the gates had already been told, was set aside so the flip-flops could have the next number. The chapters themselves are the ones you brought.

The formulas in the early drafts had been shaken apart, one glyph to a line, and then written again underneath in a hurry. They have been put back into a single line. Nothing was added to the arithmetic.

Near the end of the pile was a page of titles only: power management beyond the power chapter, firmware locks, diagnostics, and a chapter about sharing the designs. Those chapters were not written. They are named at the back, not invented here. A loose page of bootloader scraps was kept, because it was the only code the folder actually contained.

You can read it straight through, or raid the contents the way you would raid a parts bin. The first part is what a computer is, and the electrons under it. The middle is memory, a way in, and a small operating system. The last part is the wire to the next machine, and the business of keeping the one you have from cooking itself.

There is a dull warning, and it is dull on purpose. Do not connect a scavenged supply to your skin to see if it is live. Do not open a cathode-ray tube. Do not put a homemade computer on a network you do not administer, and do not treat a cipher sketch in a humor book as a lock. The undead are a metaphor. A short circuit is not.

Claudia G. Petersen

Denver, CO, 2026-09-27

PART I

The Grain of the Thing

What a computer is, before you trust it with a list.

CHAPTER 1

What Even Is a Computer



Plate 1. A briefing. The zombie is taking notes on the wrong side.

Imagine the world has gone sideways. The power's out, zombies roam your neighborhood, and your trusty laptop shortcut to the digital universe has turned into an expensive paperweight. But deep down, you know: if you can rebuild the heart of technology—the computer—you can help rebuild civilization itself.

This chapter is a no-nonsense, full-throttle expedition into what a computer is, really. Not just what it does—but how it does it, why each tiny part exists, and how to start thinking like a digital survivalist who can make one from scratch someday.

What is a Computer? At its core, a computer is a machine that:

- Takes instructions (input)
- Processes data and makes decisions (computation)
- Sends results out (output)

This sounds obvious, but like any mythic creature, it's the how that fascinates—and intimidates.

The Language of the Machine — Binary Bits

Computers speak in binary, the language of ones and zeros. Each "bit" corresponds to a small pulse of electricity: on (1) or off (0). Chains of bits spell out everything—from words on your screen to music in your headphones.

Think of the bit as a flickering campfire signal—a simple yes or no that your computer uses to build incredibly complex ideas.

The Physical Parts — Boxes, Bits, and Buttons

Your computer's hardware looks like a black box of mystery, but inside it's just:

- Tiny switches (transistors) flipping on/off
- Wires organizing pathways for electrical pulses
- Storage to hold information temporarily or forever (memory)
- The brain (CPU) that tells it what to do, faster than you can blink

Buttons and screens are the human interface—the way you shout commands into the machine or listen when it talks back.

What Actually Happens Inside? Electric impulses race along metal roads—called buses—inside the box. The CPU acts as the commander-in-chief, directing traffic, calculating sums, and making decisions in mere nanoseconds.

Programming languages, down to binary machine code, tell the CPU which switch to flip and when, in a precise dance that makes your apps run like magic.

Operating Systems: The Unsung Heroes

Operating systems (like Windows, Linux, or the tiny OS you'll build) know how to handle multitasking, memory management, file systems, and talking to your hardware. They're the critical middleman making sure your computer doesn't crash or burn every time you open a YouTube tab.

Why Zombieproofing the Computer Matters

When modern supply chains collapse and tech factories go silent, the know-how to build computers from the ground up could be the survival skill. It’s not just academic—it's life, liberty, and the pursuit of caffeinated joy.

- This book will teach you how to:
- Scavenge parts and build fundamental components
- Assemble brains and memory for your machine
- Program your creation to run real software
- Connect to peripherals and networks for expansion

By the end, you’ll not only understand computers, you’ll have the tools to build a zombieproof digital world.

CHAPTER 2

From Scavenged Bits to Building Blocks



Plate 2. Pencil lead, foil, and a lemon that has been promoted.

So, you're a survivor in a world where mass-produced electronics are relics, and factories are just spooky ruins attracting wanderers or worse. You want to build a computer from scratch — but to do that, you need to start with the basics: resistors, capacitors, diodes, transistors, and the humble wires connecting them.

This chapter isn't some sterile electronics textbook—it's your survival guide to crafting circuits in a world gone sideways. You'll learn exactly what these components do, how you can make or scavenge them, and how to bring together electricity's secrets into something that works.

What Even Is Electricity, Seriously? Electricity is a wild beast, the flow of electrons, tiny particles with a negative charge that zip through wires and materials. When you turn on a light or power a

radio, you're commanding billions of these particles to dance in sync.

Electrons flow because of a difference in electrical potential—a voltage—that pulls and pushes them, much like water flowing downhill. But unlike water, electrons obey rules set by resistance, capacitance, and other electrical properties that govern how fast, how much, and how clean the flow is.

The Components Every Circuit Needs

To harness electricity, we first need components that control and shape it:

Resistors. These are your flow controllers. Think of them as traffic cops slowing down the electron rush so your delicate devices don't get fried. You'll learn how to make your own resistors from scrap graphite or wire in a pinch.

Capacitors. The electron stashes. These little energy banks store electrical charge and release it when needed—a bit like a survival rucksack that temporarily holds vital supplies.

Diodes. One-way streets for electricity. They ensure current only flows where you want it, protecting components and crafting waves for radios and logic circuits.

Transistors. The tiny switches and amplifiers, the building blocks of modern computing magic. Made from semiconductor materials, they can open and close circuits billions of times per second, controlling flow to make decisions—the sine qua non of logic and memory.

Making Your Own Parts from Scavenged Materials

When the zombie apocalypse rolls in, microchips won't be at your local corner store (if the corner store even still exists). Crafting simple parts or scavenging from broken radios, old computers, and even kitchen appliances will be your lifeline.

DIY resistor tip. Use pencil lead (graphite) scraped onto cardboard or thin wires stretched in series to create specific resistance levels.

Capacitor crafting. Layers of aluminum foil separated by wax paper, plastic, or dry leaves make rudimentary capacitors.

Diode discovery. Find tiny glass cylinders with bands (old diodes) in junk electronics or make crude crystal diodes with razor blades and graphite points.

Wiring It Together: The Juice Line

All these parts connect together with copper wires, solder, or even improvised conductors—twisted metal, saliva-soaked cotton threads, or strips of aluminum can serve. The key is secure connections and paths for current to flow uninterrupted.

Practical Survival Project: Build a Frail but Functional LED Circuit

Materials:

- A scavenged or homemade resistor
- An LED from any broken device or a salvaged flashlight
- Power source (batteries or improvised lemon battery)
- Wire or aluminum strips
- Steps:

Connect resistor in series with LED to limit current.

Connect to power source and ground.

Observe the LED glow faintly—the first spark of hope in a dark world.

CHAPTER 3

Digital Logic from the Ground Up



Plate 3. The gate is logic. The zombie is not.

Welcome back, survivor. Now that you've learned to carve circuits from scrap and bend electrons to your will, it's time to learn the language those circuits speak: digital logic. Think of digital logic as the grammar and syntax of computing. If your computer brain is the zombieproof fortress, digital logic lays down the fundamental bricks and mortar of its walls.

Binary: The Computer's Native Tongue

Everything a computer knows collapses into two states: on or off. We call these states 1 and 0, the binary digits or "bits." This is like Morse code but electrified, where every pulse or absence of pulse carries meaning.

A bit is the foundation of all information. Group bits together and you can represent numbers, letters, images, sounds, and even ideas.

Logic Gates: The Basic Building Blocks

Logic gates are tiny, electrical decision-makers. Each takes one or more binary inputs and computes a binary output according to a rule.

The Big Six

AND: Outputs 1 if both inputs are 1. Think "both conditions must be true."

OR. Outputs 1 if either input is 1. "At least one true."

NOT: Outputs the opposite of input—flip the bit!

NAND. NOT + AND combined; outputs 0 only if both inputs are 1.

NOR. NOT + OR; outputs 1 only if both inputs are 0.

XOR. Outputs 1 if inputs differ; 0 if same. Useful for addition.

Every complex chip, every processor, and even your smartphone's brain is composed of billions of these simple gates.

Building a Half Adder — Your First Logical Circuit

Remember counting with fingers as a kid? Adding 1 + 1 equals 2, right? But computers calculate bit by bit, so adding 1 + 1 requires carrying over.

A half adder adds two single bits, **A** and **B**, and outputs a sum and a carry.

The sum is the XOR. S = A oplus B.

The carry is the AND. C = A · B.

Build it from one XOR gate and one AND gate.

You can wire this with salvaged transistors or gate ICs. Testing all four input states (00, 01, 10, 11) proves it adds correctly.

The Full Adder: Accounting for Carry-In

Life's rarely simple. Sometimes you add with a carry from previous additions.

A full adder takes in **A**, **B**, and a carry-in $\mathbf{C_{in}}$.

It outputs a sum **S** and a carry-out $\mathbf{C_{out}}$.

The formulas are **S = A oplus B oplus C_{in}** and
$\mathbf{C_{out} = (A \cdot B) + (C_{in} \cdot (A \text{ oplus } B))}$.

Build with two half adders and one OR gate.

Link multiple full adders to add multi-bit numbers—a vital function for all your zombieproof computing needs.

Oscillators and Clocks: The Heartbeat of Your Digital World

A clock isn't a hairy skull ticking—but it's just as vital. It's an electronic signal oscillating regularly between high and low voltage, marking beats that synchronize every operation.

How Do You Build a Clock? Oscillators are circuits that generate repetitive signals: sine waves, triangle waves, but for digital logic, square waves are king because they swing sharply between 0 and 1.

You can build a ring oscillator with an odd number of NOT gates connected in a loop. Each gate's output flips the input of the next, causing a continuous oscillation.

The frequency depends on gate delays—how long a signal takes to switch.

Why Square Waves? Square waves switch cleanly between high and low states, allowing digital circuits to latch onto clear logic levels without ambiguity. This sharp transition reduces errors and improves timing precision—a must for your survival-grade gear.

Clock Division

Sometimes you need a slower heartbeat. Flip-flops chained together can divide the frequency of your oscillator by powers of two, giving you adjustable timing.

Survival Project: Build a Half Adder and Clock Circuit

Parts List

Salvaged XOR, AND, NOT gates or transistors with resistors.

Breadboard or simple prototyping surface.

Power source (batteries or power supply).

LED indicators.

Steps

Wire a half adder circuit on your breadboard.

Test sum and carry outputs with all input combinations; watch the LEDs reflect the outputs.

Build a ring oscillator from NOT gates; connect an LED to observe the blinking clock signal.

Add flip-flops to divide clock frequency and observe slower blinking.

PART II

A Bit That Stays

Memory of a single moment, then a clock that makes moments into work.

CHAPTER 4

The Building Blocks Beneath the Brain



Plate 4. One bit, held gently, like a secret.

If you thought flip-flops were just dance moves, or multiplexers only a fancy tech jargon, welcome to the place where these concepts stop being mysterious names and start being tools of survival.

This chapter demystifies the crucial components that make your CPU's memory and decision paths possible, and how to build them using the logic fundamentals you already know.

Flip-Flops — The Memory Cells of Your Digital Brain

While individual logic gates make decisions right now, computers need to remember what happened a moment ago—for example, holding a bit steadily until told to change.

What Is a Flip-Flop? A flip-flop is a digital circuit that stores one bit of information—either 0 or 1—until it receives a signal to change.

How Does It Work? Flip-flops are formed by wiring logic gates (especially NAND or NOR gates) to create feedback loops:

Input signals affect the state.

At every clock pulse, the flip-flop "latches" or "captures" its input and holds that value steadily.

This ability to hold state makes flip-flops the foundation for memory and timing.

Building a Basic SR Flip-Flop

Made from two cross-coupled NOR or NAND gates.

Has Set and Reset inputs that switch the memory bit.

Simple to design but prone to forbidden states; not usually used alone in real CPUs.

The D (Data) Flip-Flop: The Workhorse

Captures the input only on clock edges, avoiding unstable states.

Easier to synchronize with the CPU clock.

Buildable from gates and SR flip-flops, forming a reliable bit-storage device.

Registers — Holding Many Bits, Holding The Line

What Is a Register? A register is a collection of flip-flops—usually 8, 16, or 32—to store multi-bit binary numbers or instructions.

How to Build a Register from Flip-Flops

Place flip-flops side by side, each storing one bit.

Add load control. when active, the register captures the input data on the clock tick. Otherwise, it holds its previous state.

Add outputs that reliably represent each bit's current value for further operations.

Why Registers Matter in Your CPU

Immediate access storage for calculations and data movements.

Fast, local compared to slower main memory.

Help form the building blocks of the CPU's scratchpad and working memory.

Shift Registers — Moving Data Bit By Bit

What Is a Shift Register? A shift register is a special kind of register that moves its bits left or right on every clock pulse, like a zipper pulling or pushing data along.

Making One from Flip-Flops

Connect flip-flops in a chain where the output of one feeds the input of the next.

On each clock tick, all bits move over by one position.

Types. serial-in serial-out, parallel-in parallel-out, etc.

Why Shift Registers Matter

Used in serial communication to convert parallel data into sequential form and vice versa.

Essential for multiplication, division, and buffering data streams.

Multiplexers — The Traffic Directors of Data

What Is a Multiplexer? Think of a multiplexer (mux) as an electronic switchboard—multiple inputs but only one output line, controlled by select signals to choose which input is routed to the output.

How Does It Work? Uses logic gates to combine inputs under select signals' control.

When select line(s) change, different input paths activate.

Enables efficient data routing without physically rewiring circuits.

Building a Multiplexer from Gates

Use AND gates to gate each input by the select lines.

Finally, OR these gated signals together into a single output.

Even an n-to-1 mux can be built from simple 2-to-1 mux blocks.

Role in the CPU

Select which register or memory data to feed into the ALU.

Control data flow during instruction execution stages.

Essential in complex decision making inside your processor.

Putting It Together — How These Parts Work in Harmony

Flip-flops hold bits steady between clock ticks.

Registers collect these bits into usable words.

Multiplexers route data dynamically based on control signals.

Shift registers move data sequentially, bit by bit, as needed.

Imagine the processor like a fortress guarded by walls (registers), doors (multiplexers), and watchtowers (flip-flops holding state). Without clear control of these, your zombieproof fortress crumbles.

Survival Project — Build a 4-bit Register and 2-to-1 Multiplexer

Materials:

- NAND or NOR gate ICs
- D flip-flop ICs or transistor versions
- Wires, breadboard, power source
- LEDs for output visualization

- Steps:

Build four D flip-flops linked into a register.

Add a load control line to enable clocked data capture.

Construct a 2-to-1 multiplexer from AND, OR, and NOT gates.

Use select input to switch between two 4-bit registers' outputs.

Test shifting, loading, and mux select changes with LEDs.

CHAPTER 5

Building the CPU Core



Plate 5. The little brain, mid-thought. The baton is not helping.

Welcome back, brave builder. Now that you've mastered the nuts and bolts of memory cells, multiplexers, and data flow control in Chapter 4a, it's time to step up to the big leagues: assembling these parts into a working CPU core, the nerve center orchestrating every move.

This chapter will guide you step-by-step through designing and wiring your own basic CPU's brain, grounding every instruction in logic gates, flip-flops, registers, and controlled data routing facilitated by multiplexers—none of the magic happens without these friends.

Understanding the CPU's Role

Your CPU is the conductor in an orchestra of circuits, maintaining rhythm with a clock and signaling which instruments—registers, memory, ALUs—play at any moment.

It performs a recurring cycle:

Fetch. Grab the instruction from memory, guided by the Program Counter (PC).

Decode. Translate that instruction into control signals that determine what the CPU does next.

Execute. Perform operation in the ALU or data transfer in registers/memory.

Writeback. Save results back to registers or memory.

Update PC. Move to the next instruction.

Bringing Circuits to Life — Step-by-Step

Step 1: Registers — Your CPU's Quick-Access Memory Registers capture and hold bits synchronized by the CPU clock:

Build each register from D flip-flops, connected to latch data loaded on clock rising edge.

Add enable (load) signals that control when the register updates, allowing precise data staging.

Use multiplexers to select which data source drives the register inputs.

Step 2: Data Movement — Controlled by Multiplexers

Imagine registers lining the CPU pathway; multiplexers are your circuit's traffic signals directing which register's data routes where.

A multiplexer logically ANDs input lines with select signals and ORs results to a single output.

Control signals from the control unit decide which register the ALU sees or what the program counter should update to.

Multiplexers let your CPU flexibly route operands and results without physically rewiring circuits.

Step 3: The ALU — The Calculating Muscle

Use the full adders and logic gates you built:

Connect registers' outputs to ALU inputs to feed numbers for arithmetic and logic operations.

Control lines select operation mode (add, subtract, and, or, xor).

Step 4: The Program Counter — Address Pioneer

The PC is a register with special logic:

Increment logic. Adder increases PC by 1 each cycle.

Load logic. When jump instructions occur, the PC loads a new address.

Controlled by multiplexers to select incremented or jumped address as next PC value.

Step 5: The Control Unit — Mastermind of Signals

This finite state machine interprets instruction bits, generating timed signals to:

Enable or disable register loading.

Select ALU operations.

Control PC increment or jump.

Manage memory read/write cycles.

Timing Is Everything: The Clock and Control Lines

Each register, ALU operation, and data movement happens on precise clock edges. Your clock pulse invades circuits, triggering flip-flops to update, multiplexers to switch paths.

Control signals gate these operations, ensuring only the correct data moves and timing stays synchronized—so no zombie shuffle or electrical chaos.

Practical Build Project: Assemble a 4-Bit CPU Core

Parts needed:

- D flip-flop ICs for registers (or transistor equivalents)
- Gate ICs/Transistors for multiplexers, logic gates, ALU adders
- Breadboard, wires, switch inputs, LEDs for outputs
- Oscillator circuit for clock source
- Procedure:

Build four 4-bit registers with controlled load signals.

Wire multiplexers to select register outputs for ALU inputs.

Construct the ALU with adders and logic gates — connect to multiplexers' outputs.

Build the program counter with increment and load logic, controlled by multiplexers.

Create a minimalist control unit hardwired for a few basic instructions: load, add, jump, store.

Connect clock to synchronize all components.

Test with step-by-step clock pulses, watching LEDs for register outputs and program counter progress.

This grounding completes your CPU's brain, built from first principles, ready to be expanded with memory, input/output, and complex instructions. By controlling every gate, flip-flop, multiplexer, and wiring path, you hold the power to resurrect civilization's digital pulse.

PART III

Something You Can Use

A place for the program, a way to touch the machine, and a small system between you and the metal.

CHAPTER 6

Memory Integration



Plate 6. Drawers for thoughts. Most of them empty, which is honest.

Welcome to the most crucial layer of your homemade zombieproof machine: memory. Without memory, your processor is a headless survivor, unable to remember instructions, data, or even which step it's on. Memory is both the filing cabinet and scratchpad for your digital brain.

This chapter is a deep dive into memory architecture, address decoding, control signals, data buses, and how to hook up your CPU to real memory chips or banks. You'll learn all the seemingly mystifying details of making memory work seamlessly so your CPU can focus on surviving, computing, and... well, not getting eaten.

Why Memory Integration Matters

Your CPU needs memory for several critical tasks:

Store programs (instructions).

Hold variables and data during operations.

Provide workspace for intermediate calculations.

Without memory, all the logic in the world serves no purpose. The CPU would just stare into empty electrical void.

Memory Types—Basics You Need to Know

RAM (Random Access Memory): Temporary and fast storage, erased when power is off—your prime workspace. Built as arrays of flip-flops or capacitors arranged in grids.

ROM (Read-Only Memory). Permanent storage holding essential firmware or bootstrapping code. Non-volatile, but harder to modify.

Memory-Mapped I/O. Peripherals appear as memory addresses — simplifying CPU access to devices.

Address Spaces and Address Lines

Memory chips are organized in locations accessible by unique addresses. The number of address lines dictates how much memory you can access:

N address lines give 2^N unique addresses.

For example, 8 address lines = 256 addresses; 16 = 65,536.

Your CPU sends address info down the address bus— a set of wires dedicated to carrying address bits.

Decoding Addresses: The Key to Organized Access

With many memory chips and devices, your CPU must ensure only one device answers per address. This calls for decoders or address decoders.

These are logic circuits (made from NAND, NOR gates, or multiplexers) that listen to specific address bits.

When the CPU address matches a chip's range, the chip decoder outputs an enable signal to activate the chip.

This prevents multiple chips trying to drive the bus at once, which would cause data corruption or electrical damage.

Data Bus and Control Signals — The Communication Layer

The data bus is a multi-wire path carrying actual data to/from memory and CPU.

Control signals include:

Read/Write (R/W). Whether data is being read from or written to memory.

Chip Enable (CE). Permits a specific chip to engage and place data onto the bus.

Output Enable (OE). Makes sure only selected devices drive the bus.

Tristate Buffers — Keeping the Peace on the Bus

When multiple memory chips share a data bus, tristate buffers ensure only one device drives the lines at a time, while others stay 'off' electrically.

Practical Integration Walkthrough

CPU puts the target memory address on the address bus.

Address decoder activates the matching chip's enable signal.

CPU asserts Read or Write.

During Read, the selected chip drives the data bus with requested data.

During Write, the CPU places data on the bus, and the chip saves it.

Timing is critical. Control signals and data must stabilize before reads/writes are acknowledged.

Survival Project: Building a Basic Memory Interface

Materials RAM IC or homemade flip-flop arrays for memory banks.

NAND gates or decoders for chip select circuitry.

Tristate buffers or logic gates for bus management.

Wires, breadboards, connection tools.

Procedure

Design address decoding for your available RAM banks or modules.

Wire control lines for read and write operational modes.

Connect tristate buffers to ensure clean data bus sharing.

Demonstrate reading and writing sequences with LEDs or logic analyzers.

Program a small loop or test code inside your CPU to read from and write to memory.

Timing, Delays, and Real World Imperfections

Propagation delays mean data doesn't move instantly; clock synchronization and wait states make sure signals settle.

Power stability and noise immunity are practical concerns—buffer your signals.

Consider handshaking protocols for perfect data transfer with external memory devices.

Memory integration is a mighty challenge and opportunity — the difference between a theoretical CPU and a fully functioning computer that might just reboot civilization.

CHAPTER 7

Peripherals and Serial Communication



Plate 7. The mouse has seen things. It still clicks.

The brains inside your zombieproof computer crack problems and spit out solutions, but it doesn't know a thing about the zombies, your hands, or the world outside unless you give it senses and limbs. Those come via peripherals: keyboards, mice, monitors, speakers—all the gadgets that let you feed data in and get results back out.

This chapter equips you with a down-and-dirty, no-nonsense look at peripherals, their communication protocols, and practical projects like building a working mouse interface from scratch.

The Basics of Peripherals: Input and Output Explained

Input Devices At their core, input devices convert physical user actions into electrical signals your computer can understand.

Keyboards. Pressing a key sends a scan code to the CPU, often over serial or parallel interfaces.

Mice. Capture motion and button presses, encoding them as digital pulses.

Switches and sensors. Simple inputs that toggle bits 'on' or 'off'.

Output Devices

Output devices translate your computer's digital signals into the physical or graphical world.

LEDs. Simple binary indicators, on and off controlled by bits.

Displays. From simple 7-segment LEDs to complex pixel arrays, they transform data into visuals.

Speakers. Convert digital audio data into sound waves.

Serial Communication: The Lifeline Protocol for Peripherals

Serial data transfer sends information one bit at a time over a wire or pair of wires—a bit like sending Morse code. It is the backbone for many peripherals because it simplifies wiring and is bandwidth-efficient for user input devices.

Key Serial Protocols

UART (Universal Asynchronous Receiver/Transmitter): Asynchronous serial communication used by many simple devices.

SPI (Serial Peripheral Interface). A synchronous protocol for short-distance, fast communication, e.g., sensors.

I2C (Inter-Integrated Circuit). Multiple devices sharing two wires, each with a unique address.

How Serial Communication Works Practically

Data bits are clocked out one by one.

Start and stop bits frame each byte.

Parity bits can check data integrity.

Both devices must agree on baud rate (bits per second).

Building a Zombieproof Mouse — From Parts to Protocol

Here is a practical breakdown:

How a Mouse Works Mechanically

Motion detection: Optical sensors track surface movement, turning it into x/y data increments. Mechanical mice use roller balls with encoders.

Button switches. Simple contacts that close circuits when clicked.

How a Mouse Communicates

Mouse data packets are serially sent to the computer.

Standard protocols like PS/2 or USB structure these packets with defined bytes for movement and button states.

Serial nature allows simple two-wire connections and relatively low hardware complexity.

Building Your Own Mouse

Use optical or mechanical sensors scavenged from old devices.

Implement a simple microcontroller or logic circuit to buffer movement data and button presses.

Encode data packets according to a serial protocol.

Transmit via UART or similar serial line to your homemade CPU.

Practical Survival Project: Serial Mouse Interface

Parts

- Optical sensor assembly or rotary encoders
- Push-button switches
- Logic gates or simple microcontroller for encoding
- UART compatible output stage
- Breadboard, wiring, power
- Steps

Assemble sensors to detect motion increments, convert analog signals to pulses.

Attach buttons as digital switches with debouncing circuits.

Create simple state machine or logic circuit to format and serialize data packets.

Connect to CPU serial input port, verifying data integrity and timing.

Develop CPU-side code or hardware interpreters to translate mouse packets into cursor movements or commands.

Other Important Peripherals: Keyboards and Displays

Keyboards scan key matrices, sending codes serially to the computer; usually use protocols similar to mice.

Displays need framebuffer storage and protocol to update pixels or segments; simple LED arrays can be driven directly by CPU via multiplexers.

Consider how interrupt-driven input can optimize responsiveness.

Bringing peripherals online with serial communication protocols is the key to making your homemade computer a practical, interactive survivor ready for the end times.

CHAPTER 8

Simple Operating Systems

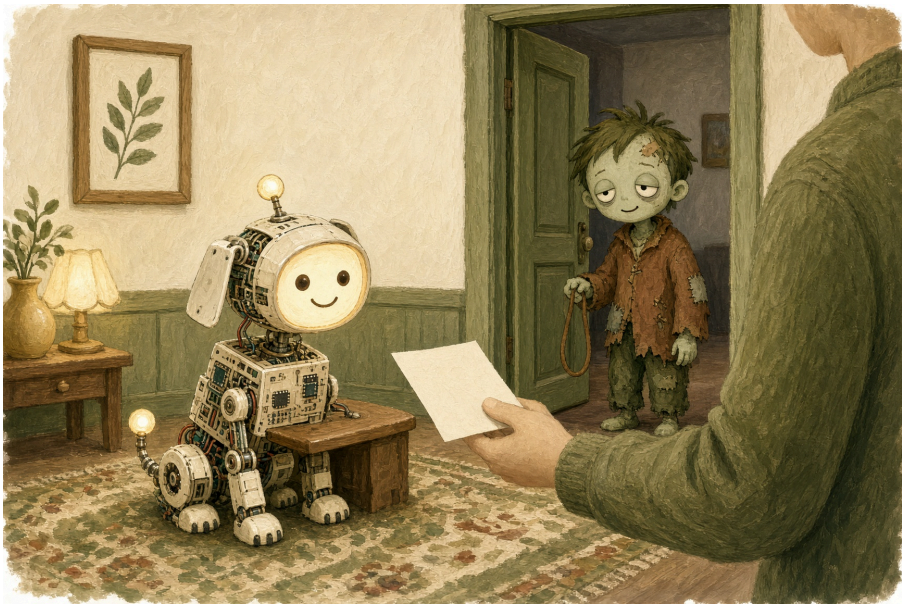


Plate 8. Someone has to tell the hardware what day it is.

So you've built a CPU, hooked up memory, and connected peripherals. Congratulations, you now have the rudiments of a functional computer—just like the mythical devices whispered of in pre-apocalypse times! But raw hardware is like a wild dog: it needs training and organization to become your loyal companion.

Enter the operating system (OS)—the indispensable software layer that manages the hardware, runs programs, and provides a user-friendly way to interact with the machine. This chapter explores what operating systems are, why they're essential, and guides you in creating a simple OS to bridge your hardware and your survival needs.

What Is an Operating System? An OS is a collection of software routines that:

- Manage hardware resources like CPU, memory, and I/O devices
- Provide a way to load, run, and manage programs

- Facilitate communication with input/output devices
- Handle file storage, execution control, and error management

Without an OS, users must command the machine directly at the binary level—a headache of cosmic proportions.

OS Components in a Nutshell

Bootloader: Tiny initial program that prepares the CPU and memory to load the OS itself.

Kernel. The core that talks directly to hardware and manages resources.

File System. Organizes storage so you can save and retrieve data easily.

Device Drivers. Specialized code to communicate with peripherals.

Shell/User Interface. Allows users to run commands, start programs, and run scripts.

Building a Minimal OS for Your Zombieproof Computer

Step 1: The Bootloader Stored in ROM or initial memory region.

Initializes system state, sets up stack, and loads kernel from memory or storage into RAM.

Step 2: Kernel Design

Handles interrupts — special signals from hardware peripherals or timers.

Implements simple scheduler to switch between program tasks.

Provides basic memory management and input/output control.

Step 3: Simple Shell

Command line interface to execute programs, list files, and control peripherals.

Support basic commands like run, load, save, and exit.

How to Write OS Code

Your OS will be written in low-level languages such as assembly or C-like code, carefully tailored to your CPU's instruction set.

Start small, with routines to initialize hardware and respond to keyboard input.

Use interrupts to handle asynchronous events like ticking clocks and peripheral signals.

Implement basic file handling if your system has storage.

Survival Project: Writing and Loading a Tiny OS

Write bootloader to initialize hardware and load kernel.

Develop kernel to respond to keyboard and timer interrupts.

Create shell for basic file handling and program execution.

Test system by running simple user programs (print text, echo input).

Why Your Zombieproof Computer Needs an OS

Automates routine tasks, so you can focus on survival strategies, not flipping bits manually.

Abstracts hardware complexities for better modular programming and expansion.

Enables multitasking and efficient resource management—critical when zombies are knocking.

Operating systems are the glue and brain interface between raw hardware and practical use. By building one tailored to your design, you gain full command over your zombieproof computer's

capabilities.

PART IV

The Next Machine

A wire to an ally, and the whole build laid out in one order.

CHAPTER 9

Networking Your Zombieproof Computer



Plate 9. Two machines, and one polite wire between them.

You've built a rugged digital fortress—CPU, memory, and peripherals all working in harmony. But in the zombie apocalypse, even the strongest lone survivor benefits from allies, intel networks, and coordinated defense. Networking connects your machines into a resilient tribe, sharing the battle-worn load.

This chapter gives you the science-backed, deep dive into adding networking capabilities. It covers fundamental concepts, practical wiring, protocols, and a survival-ready example of linking multiple zombieproof machines.

Why Network? Networking lets multiple computers:

- Share data and resources, increasing efficiency
- Collaborate on tough tasks beyond one CPU's strength
- Exchange real-time info, warnings, and coordinated commands

- Form the backbone of rebuilding post-apocalyptic infrastructure

The Physical Layer — Networking Hardware

Wired Connections Simple serial cables between machines

- Ethernet-style twisted pairs (complex but powerful)
- USB or similarly structured digital buses

Wireless Options

Radio signals like Wi-Fi or Bluetooth (power-heavy, intricate protocols)

Network Interfaces — Your Machine's Network ID Card

Each device needs a network interface:

- Converts internal data to signals for the physical medium
- Receives signals, converts back to data
- Manages collisions, timing, and error detection

In DIY setups, UART or SPI modules often act as simple interfaces.

Data and Packets: The Language of Networked Machines

Networks communicate by breaking data into packets—discrete units framed with:

Headers: Destination and source addresses, control info

Payload: Actual data content

Error Detection: Checksums or CRCs for detecting corruption

Protocols — Communication Rules

Just like survivors agree on signals (lights, sounds), machines agree on protocols for talking:

Simple serial protocols: RS-232, UART—easy for small-scale connections

TCP/IP: Complex internet backbone protocol

Custom lightweight protocols: Designed for specific machine-to-machine communication

Your zombieproof rigs will likely start with simple serial links expanding to more complex protocols as needed.

Example: Point-to-Point Serial Network

Wiring two zombieproof machines with serial TX/RX lines:

- Use UART communication, with matching baud rates and framing
- Simple flow control to avoid data loss (wait for ACKs or use buffers)
- Software on both ends handles packet formation and interpretation

Survival Project: Building Your Network Interface

Materials:

- UART modules or build UART logic from gates
- RS-232 level shifters or transistor circuits for signal integrity
- Cables and connectors
- Basic logic analyzer or LED indicators for debugging
- Steps:

Wire serial TX of one CPU to RX of another; cross-connect lines.

Implement baud rate generators for timing the bits.

Develop software to encode messages, add checksums.

Write software to decode and respond to messages.

Debug with LEDs and logic probes; ensure proper data flow.

Networking your zombieproof computer turns isolated survivor devices into a strong, coordinated network. Together, you can outsmart zombies and reboot civilization with real, networked machines communicating reliably over homemade links.

CHAPTER 10

A Hands-On Blueprint



Plate 10. Lay the kit out before the night does it for you.

At this stage At this stage, you've gained a deep understanding of how computers think and work, memory integration, peripherals, operating systems, and networking for survival scenarios. Now it's time to bring all that knowledge together into a cohesive, step-by-step project guide. This chapter lays out a complete blueprint for building a functional, zombieproof computer from the ground up.

Step 1: Designing Your CPU and Basic Logic

Circuit choices: Use discrete logic gates (NAND, NOR, XOR) or programmable logic devices depending on your resources.

Clock generation. Build a stable oscillator circuit, possibly using inverter loops or crystal oscillators.

Registers and flip-flops. Assemble data storage bit cells from D flip-flops built from NAND gates.

ALU. Construct arithmetic and logic units with adders, multiplexers, and basic logic circuits.

Step 2: Memory Setup

RAM modules: Make or source RAM chips; design address decoders for mapping.

ROM modules. Store bootloader and firmware for system startup.

Bus design. Wire address, data, and control buses with tri-state buffers to prevent conflicts.

Step 3: Input/Output and Peripheral Interface

Peripherals: Build simple input devices (switches, keyboard matrix) and output devices (LEDs, 7-segment displays).

Serial communication. Implement UART-based serial ports to handle communication with mice, keyboards, or additional zombieproof rigs.

Device drivers. Develop minimal code routines to operate peripherals.

Step 4: Operating System Basics

Develop bootloader and kernel to initialize hardware, manage interrupts, and schedule programs.

Create a basic shell for command execution.

Implement simple file storage and program loading processes.

Step 5: Networking Your System

Wire UART serial links between two or more zombieproof computers.

Establish communication protocols with framing, error checking, and handshake mechanisms.

Develop software layers to send and receive data packets.

Step 6: Testing, Debugging, and Expansion

Use LEDs, logic analyzers, and oscilloscopes to monitor signals.

Debug hardware and software modules incrementally.

Add expansion capabilities. more memory, complex peripherals, advanced OS features.

Survival Project Wrap-Up and Motivation

Building a fully functional computer from scratch in a resource-constrained, zombie apocalypse environment is challenging but achievable with patience, discipline, and the strategies laid out in this guide. By understanding every component's why and how, you not only build a machine—you forge resilience for the new world.

PART V

Heat, Hunger, and Mistakes

The bugs, the volts, and the box that has to live in weather.

CHAPTER 11

Debugging and Optimizing



Plate 11. The probe is mightier than the speech.

Building your own zombieproof computer is like crafting a fortress out of scavenged parts while zombies surround you—every connection, circuit, and line of code needs meticulous care. This chapter is your survival manual for debugging hardware and software, optimizing designs, and preventing apocalypse-level failures.

Understanding Common Bugs and Their Origins

Hardware glitches: Loose connections, noise, timing violations, power supply fluctuations.

Logic errors. Incorrect gate wiring, signal contention on buses, race conditions in flip-flops.

Software bugs. Infinite loops, incorrect interrupt handling, memory corruption.

Hardware Debugging Tools and Techniques

Multimeter: Check continuity, voltage levels, and verify power rails.

Oscilloscope. Observe signal waveforms, timing, clock stability, and investigate glitches.

Logic analyzer or LED indicators. Monitor digital signals and data buses.

Test benches. Isolate components and test subsystems independently.

Debugging Strategies

Modular testing: Verify each component individually before system integration.

Incremental integration. Assemble and test small subsystems progressively.

Signal tracing. Follow signals through your circuit with an oscilloscope or probes to find breakdowns.

Code walkthroughs. Step through your OS and firmware with careful review and debug prints/outputs.

Optimizing for Real-World Survival

Minimize power consumption: Use low-power logic families, clock gating, and efficient code.

Noise immunity. Shield cables, use clean power supplies, and proper grounding.

Heat management. Adequate ventilation and low-current components.

Scalability. Design modular circuits allowing easy upgrades or repairs in the field.

Survival Exercises

Build test circuits for critical components (ALU, registers) and verify function using LEDs or logic analyzers.

Create a simple firmware watchdog timer to recover from software hangs.

Analyze and fix timing issues by adjusting clock speed and signal routing.

Mastering debugging and optimization assures your zombieproof machine isn't just functional but tough and reliable, ready to survive long-term apocalypse challenges.

CHAPTER 12

Powering Your Zombieproof Computer



Plate 12. Power, which is a polite word for not going dark.

In the apocalypse, your machine's lifeblood is electricity—without it, no CPU, memory, or peripherals come to life. This chapter covers how to design a robust power supply for your zombieproof computer, handling voltage regulation, power stability, and energy sources that keep your digital fortress running no matter the external chaos.

Power Sources — Picking Your Fuel

Batteries: Portable but limited runtime; choose types with good energy density like Li-ion or NiMH.

Solar panels. Sustainable if you can catch the light; requires batteries or capacitors for buffering.

Generators. Mechanical but noisy and fuel-dependent; good for basecamp setups.

Hand-crank or pedal power. Emergency use for short bursts.

Voltage Regulation

CPUs and logic chips require clean, steady voltages (usually 5V, 3.3V, or 1.8V).

Use linear regulators or switch-mode power supplies (buck converters) for efficient steady outputs.

Add capacitors for smoothing to filter out voltage spikes and noise.

Power Distribution and Protection

Design power buses to distribute voltage and ground properly.

Use fuses or resettable polyfuses to prevent damage from shorts.

Implement decoupling capacitors near ICs for local stability.

Managing Power Efficiency

Minimize consumption by using low-power components.

Use sleep/standby modes in firmware to shut down unused parts.

Turn off clock feeds where possible (clock gating).

Survival Project: Building a Reliable Power Supply Unit

Gather batteries, voltage regulators, capacitors, and wiring.

Assemble a circuit converting battery voltage to stable 5V or 3.3V rails.

Add indicator LEDs to show power status.

Test under load with your assembled boards.

Powering your computer with confidence is the foundation for all other zombieproof designs.

CHAPTER 13

Cooling and Enclosure



Plate 13. A box that lets the heat out and the weather stay outside.

Now that you've mastered brainpower, memory, and power supply, it's time to keep your zombieproof computer physically safe from the elements and overheating—both of which can kill even the toughest hardware quicker than a zombie bite.

This chapter covers practical cooling techniques, enclosure design principles, and durability hacks tailored for survival scenarios, so your machine keeps firing on all cylinders when the undead horde comes knocking.

Why Cooling Matters

Digital circuits generate heat due to current flow through millions of transistors and logic gates.

Excess heat can cause timing errors, degrade components, and shorten electronics lifespan.

Efficient heat dissipation is vital for reliable operation, especially in confined spaces or hot environments.

Basic Cooling Techniques

Natural convection: Design enclosures with ventilation holes allowing airflow.

Passive cooling. Use heatsinks—metal fins that draw heat away from chips.

Active cooling. Small fans powered by your system, moving air to remove heat.

Thermal interface materials. Use thermal paste or pads to improve contact between chips and heatsinks.

Enclosure Design Principles

Use durable, non-conductive materials (plastic, wood treated for flame resistance).

Make it modular for easy access to boards and components for repair.

Protect from dust, water, and shock with sealed or padded compartments.

Include mounting points for cables, sensors, and peripherals securely.

Survival Enhancements

Use surge protectors and power conditioners against erratic power sources.

Implement humidity and temperature sensors to alert for dangerous conditions.

Consider camouflage enclosures for stealth in hostile territories.

Survival Project: Assembling a Durable Enclosure

Survey available materials — repurposed casings, wood boxes, rugged plastics.

Design airflow paths balancing protection and cooling.

Integrate power and interface pass-throughs with sealed connectors.

Test enclosure with temperature sensors under extended operation.

A well-cooled, well-protected zombieproof computer will outlast threats both natural and undead, empowering you with dependable tech for survival.

PART VI

Words the Machine Keeps

A net you can trust a little, code that fails politely, and a path around the processor when the data will not wait.

CHAPTER 14

Expanding and Securing the Network



Plate 14. Trust, but checksum.

You've got a functioning zombieproof computer, powered, cooled, and even talking with its peers. Now it's time to think big—expanding your network and securing it against threats from both undead hackers and environmental damage.

This chapter explores scalable network topologies, advanced protocols for robust communication, plus encryption and access control for safe and survivor-approved digital exchanges.

Network Scalability

Mesh networks: Each node relays data, creating robust, redundant paths.

Star networks. Central node controls communication; easier but riskier.

Hybrid models. Combine mesh and star for balance of efficiency and resilience.

Advanced Protocols

Error correction codes (ECC) for reliable data in noisy environments.

Retransmission strategies and acknowledgments to guarantee delivery.

Routing protocols to handle multiple hops and network partitions.

Security Fundamentals

Encryption: Use simple symmetric ciphers or custom schemes for confidentiality.

Authentication. Devices verify each other with shared keys or challenge-response.

Access control. Restrict network resource use to trusted nodes only.

Survival Project: Securing Your Network

Implement handshake protocols ensuring only authorized devices join.

Encrypt messages with lightweight algorithms suitable for limited resources.

Log network events and monitor for suspicious activity with simple counters.

Environmental Hardening

Protect network cables and nodes against moisture, temperature, and shock.

Use ruggedized connectors and sealed enclosures.

Plan for redundant power supplies and backup communication lines.

With a secure, scalable network, your zombieproof computer community becomes a lifeline in the apocalypse—sharing knowledge, resources, and digital firepower.

CHAPTER 15

Writing Resilient Survival Code



Plate 15. Code small enough to carry in your head.

With your hardware built, powered, networked, and cooled, it's time to wake your zombieproof computer with software designed to thrive in apocalypse conditions. Writing resilient, resource-efficient code is essential to keep your machine reliable and useful when survival depends on it.

This chapter dives into low-level programming, handling hardware interrupts, writing bootloaders, simple operating systems, and network communication code—all infused with survival-minded best practices.

Getting Started with Low-Level Programming

Use Assembly or systems languages like C or Rust for fine-grained control.

Understand your CPU's instruction set, registers, and memory map thoroughly.

Write small, modular routines to control CPU, memory, and I/O devices.

Interrupt Handling

Interrupts allow your machine to respond immediately to events (timers, keyboard, network).

Write interrupt service routines (ISRs) to handle signals quickly and safely.

Manage interrupt priorities and masking to avoid conflicts.

Writing Bootloaders

Your bootloader is the first code to run, setting up the CPU and loading the OS or main program.

Keep it small, simple, and reliable—test thoroughly.

Handle hardware initialization, stack setup, and jumping to OS entry points.

Operating System Basics

Design minimal kernels to manage tasks, memory, and device drivers.

Implement simple shells or command interpreters for user interaction.

Add file storage and loading features if hardware permits.

Network Communication Code

Write routines for sending and receiving serial packets with framing and error checks.

Implement handshake and retransmission for reliability.

Consider lightweight encryption for message security.

Survival Coding Practices

Code defensively: check inputs, handle errors gracefully, avoid infinite loops.

Conserve resources. use minimal data structures and avoid busy waiting.

Test with simulated hardware faults and noisy inputs.

Survival Project: Bootloader and Shell

Write a simple bootloader that initializes your zombieproof CPU and memory.

Develop a shell capable of basic commands—display text, list files, run programs.

Extend with network commands to send messages or control remote rigs.

Programming your zombieproof computer with survival-aware software unlocks the full power and reliability of your custom build—not just a machine, but a resilient digital companion in the apocalypse.

CHAPTER 16

A Bootloader and a Shell



Plate 16. It boots. It asks what you want. That is a civilization.

Here are some practical programming examples and survival tips for your zombieproof computer's bootloader and shell:

Bootloader Example (Assembly Pseudocode)

```
; Initialize stack and memory
start:
  ldi sp, STACK_TOP    ; Load stack pointer
  call init_memory     ; Setup memory mapping
  call init_devices    ; Setup peripherals

  ; Load main program from ROM to RAM
  call load_program

  ; Jump to main program entry point
  jmp main_entry
```

Simple Shell Command Example (C)

```
void shell_loop() {
  while (1) {
    print_prompt();
    char cmd[64];
    read_input(cmd, sizeof(cmd));

    if (strcmp(cmd, "help") == 0) {
      print_help();
    } else if (strcmp(cmd, "run") == 0) {
      run_program();
    } else if (strcmp(cmd, "exit") == 0) {
      break;
    } else {
      print("Unknown command\n");
    }
  }
}
```

Survival Tips for Programming

Keep code minimal and modular for easy updates and debugging.

Avoid blocking calls; use interrupts or event-driven design.

Work in small increments; test frequently on your hardware.

Use checksums and validation on data transfers to avoid corruption.

Log errors or status with LEDs or serial output for offline diagnosis.

CHAPTER 17

Memory Management and Direct Memory Access



Plate 17. The long way around the processor, on purpose.

To truly survive and help build civilization anew, your zombieproof computer needs to handle memory efficiently and move data rapidly without choking the CPU. This chapter delves into the design of memory management units, direct memory access (DMA), and techniques that optimize throughput.

Memory Management Principles

Memory mapping: Assign fixed or dynamic addresses to memory regions, peripherals, and I/O devices.

Paging and segmentation (simplified). Organize memory to separate code, data, stack, and peripherals for safety and ease.

Address decoding circuits. Hardware logic to select correct memory blocks based on CPU address lines.

Direct Memory Access (DMA)

DMA controllers enable peripherals to read/write main memory independently, reducing CPU load.

Ideal for high-speed data transfers (disk I/O, network packets, video buffers).

Requires careful management of bus arbitration to avoid conflicts.

Designing DMA in Zombieproof Machines

Build simple DMA logic with finite state machines that handle memory access cycles.

Use interrupts or status registers to signal completion of transfers.

Ensure synchronization mechanisms between CPU and DMA to prevent data corruption.

Survival Optimization

Add memory protection to prevent buggy code from overwriting critical regions.

Design for minimal power consumption in memory cycling and DMA operations.

Use buffers to smooth out bursty data loads.

Survival Project: Implementing Basic DMA

Design state-machine logic for DMA read/write cycles.

Program CPU to initiate DMA transfers and handle completion interrupts.

Test with block data transfers between I/O and RAM, verifying integrity.

Mastering memory management and DMA equips your zombieproof computer with smart data flows, freeing up CPU cycles and making complex tasks survivable under apocalypse constraints.

Afterword

The last chapter of a computer book is a temptation to get solemn about the future. Resist it a little. The bench is not a chapel. It is a table you share with whoever needs the machine to remember a thing, and the courtesy that keeps it usable is older than any of us: label the wire, check the rail before you blame the code, and leave a way to boot again after you have been foolish.

If you have read this far you already know the unglamorous truth. A processor without memory is a mood. A memory without a clock is a rumor. The question here is not how far a signal goes. It is whether the next person can turn the machine on and find the work still in it. The person who brings a spare regulator will be remembered more kindly than the person who brings a speech.

The zombies, if you still need them, are only the name we gave the interruption. A brownout is one. A data bus with two drivers is one. Panic that rewrites the bootloader is one. You do not beat them by adding features. You beat them by being able to explain, on paper, what the box is supposed to do when the light comes back.

Leave a note on the lid. Tell someone else how it boots. When you are done for the night, say so, and power it down on purpose, so nobody sits up watching a cursor out of loyalty.

The signature under an afterword belongs to the person who stands by the book. That line is blank until you want it.

Save your work. Then quiet.

Claudia G. Petersen

Manhattan, NY, 2026-10-02

Acknowledgments

Eternal thanks to Petey, NotebookLM, Purple Exity, Gemini and the endless hoardes of preppers 'n off grid type peoples out there who share.

About the author

Claudia G. Petersen

Claudia G. Petersen is a CEO - twice, most recently a software dev, but in previous lives an EE, a Genetic Engineer, an Engineering Microbiologist, and, on occasion, an Author.

Also from whispr written word

Zombie Proof: Looting

Finding and recovering useful supplies after the collapse.

Zombie Proof: Fortifying

Making shelter harder to enter and easier to defend.

Zombie Proof: Weapons

Weapons, their limitations, and the responsibility of handling them.

Zombie Proof: Morale & Psychology

Keeping your head and keeping people together when the world falls apart.

Zombie Proof: Medicine

Managing injuries, medical supplies, and illness when infrastructure fails.

Colophon

Zombieproof Computers: How to Keep a Mind When the Grid Is Gone was set in Fraunces and Literata. The plates were drawn for this edition. The text was prepared from the chapter drafts. Duplicate passes were set aside, the shaken-apart formulas were put back on one line, and the file-crumbs were swept off the bench.

Trim: six by nine inches. First edition.

If you are holding a file and not a book, print it on paper that can stand a workbench. Leave the last page near the machine, not in a drawer.



whispr written word

Denver, CO